

last time

TLB: VPN $\rightarrow$ PTE cache

TLB tags and indices

threads and shared {open files, memory}

pthread_create $\sim$ fork

pthread_join $\sim$ waitpid

quiz Q6

main() creates thread running foo()

both have p1 local variable

stored on stack

since two stacks in same process → different virtual addresses

reminder re: pagetable2

strict deadline **before first lab** (3:29pm)

code review activity

- not debugging; code organization/style feedback
- showing code on your screen, not sending files

sum example (only globals)

```
int values[1024]; int results[2];
void *sum_front(void *ignored_argument) {
    int sum = 0;
    for (int i = 0; i < 512; ++i) { sum += values[i]; }
    results[0] = sum;
    return NULL;
}
void *sum_back(void *ignored_argument) {
    int sum = 0;
    for (int i = 512; i < 1024; ++i) { sum += values[i]; }
    results[1] = sum;
    return NULL;
}
int sum_all() {
    pthread_t sum_front_thread, sum_back_thread;
    /* missing: error handling */
    pthread_create(&sum_front_thread, NULL, sum_front, NULL);
    pthread_create(&sum_back_thread, NULL, sum_back, NULL);
    pthread_join(sum_front_thread, NULL); pthread_join(sum_back_thread, NULL);
    return results[0] + results[1];
}
```

sum example (only globals)

```
int values[1024]; int results[2];  
void *sum_front(void *ignored_argument) {  
    int sum = 0;  
    for (int i = 0; i < 512; ++i) { sum += values[i]; }  
    results[0] = sum;  
    return NULL;  
}  
void *sum_back(void *ignored_argument) {  
    int sum = 0;  
    for (int i = 512; i < 1024; ++i) { sum += values[i]; }  
    results[1] = sum;  
    return NULL;  
}  
int sum_all() {  
    pthread_t sum_front_thread, sum_back_thread;  
    /* missing: error handling */  
    pthread_create(&sum_front_thread, NULL, sum_front, NULL);  
    pthread_create(&sum_back_thread, NULL, sum_back, NULL);  
    pthread_join(sum_front_thread, NULL); pthread_join(sum_back_thread, NULL);  
    return results[0] + results[1];  
}
```

values, results: global variables — shared

sum example (only globals)

two different functions

happen to be the same except for some numbers

```
int values[1024];  
void *sum_front(void *ignored_argument) {  
    int sum = 0;  
    for (int i = 0; i < 512; ++i) { sum += values[i]; }  
    results[0] = sum;  
    return NULL;  
}  
void *sum_back(void *ignored_argument) {  
    int sum = 0;  
    for (int i = 512; i < 1024; ++i) { sum += values[i]; }  
    results[1] = sum;  
    return NULL;  
}  
int sum_all() {  
    pthread_t sum_front_thread, sum_back_thread;  
    /* missing: error handling */  
    pthread_create(&sum_front_thread, NULL, sum_front, NULL);  
    pthread_create(&sum_back_thread, NULL, sum_back, NULL);  
    pthread_join(sum_front_thread, NULL); pthread_join(sum_back_thread, NULL);  
    return results[0] + results[1];  
}
```

sum

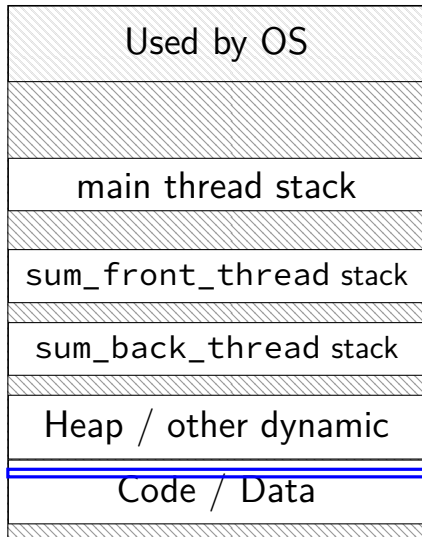
values returned from threads
via global array instead of return value
(partly to illustrate that memory is shared,
partly because this pattern works when we don't join (later))

```
int values[1024];
void *sum_front(void *ignored_argument) {
    int sum = 0;
    for (int i = 0; i < 512; ++i) { sum += values[i]; }
    results[0] = sum;
    return NULL;
}

void *sum_back(void *ignored_argument) {
    int sum = 0;
    for (int i = 512; i < 1024; ++i) { sum += values[i]; }
    results[1] = sum;
    return NULL;
}

int sum_all() {
    pthread_t sum_front_thread, sum_back_thread;
    /* missing: error handling */
    pthread_create(&sum_front_thread, NULL, sum_front, NULL);
    pthread_create(&sum_back_thread, NULL, sum_back, NULL);
    pthread_join(sum_front_thread, NULL); pthread_join(sum_back_thread, NULL);
    return results[0] + results[1];
}
```

thread_sum memory layout



0xFFFF FFFF FFFF FFFF

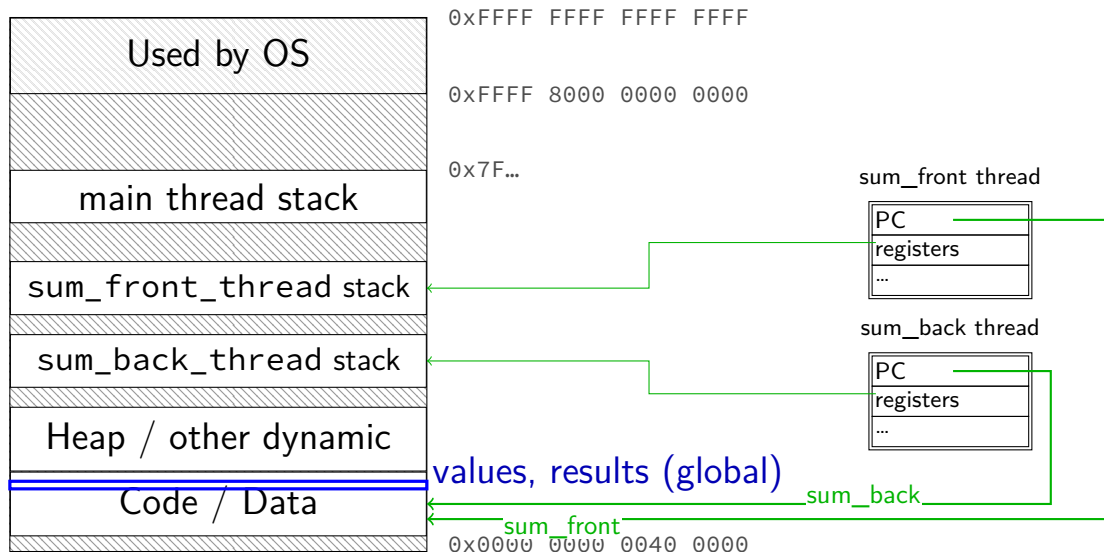
0xFFFF 8000 0000 0000

0x7F...

values, results (global)

0x0000 0000 0040 0000

thread_sum memory layout



sum example (to global, with thread IDs)

```
int values[1024];
int results[2];
void *sum_thread(void *argument) {
    int id = (int) argument;
    int sum = 0;
    for (int i = id * 512; i < (id + 1) * 512; ++i) {
        sum += values[i];
    }
    results[id] = sum;
    return NULL;
}
int sum_all() {
    /* missing: error handling */
    pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        pthread_create(&threads[i], NULL, sum_thread, (void *) i);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return results[0] + results[1];
}
```

sum example (to global, with thread IDs)

```
int values[1024];
int results[2];
void *sum_thread(void *argument) {
    int id = (int) argument;
    int sum = 0;
    for (int i = id * 512; i < (id + 1) * 512; ++i) {
        sum += values[i];
    }
    results[id] = sum;
    return NULL;
}
int sum_all() {
    /* missing: error handling */
    pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        pthread_create(&threads[i], NULL, sum_thread, (void *) i);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return results[0] + results[1];
}
```

values, results: global variables — shared

sum example (info struct)

```
int values[1024];
struct ThreadInfo {
    int start, end, result;
};
void *sum_thread(void *argument) {
    struct ThreadInfo *my_info = (struct ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) { sum += values[i]; }
    my_info->result = sum;
    return NULL;
}
int sum_all() {
    pthread_t thread[2]; struct ThreadInfo info[2];
    for (int i = 0; i < 2; ++i) {
        info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, &info[i]);
    }
    for (int i = 0; i < 2; ++i) { pthread_join(threads[i], NULL); }
    return info[0].result + info[1].result;
}
```

sum example (info struct)

```
int values[1024];
struct ThreadInfo
    int start, end, result;
};
void *sum_thread(void *argument) {
    struct ThreadInfo *my_info = (struct ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) { sum += values[i]; }
    my_info->result = sum;
    return NULL;
}
int sum_all() {
    pthread_t thread[2]; struct ThreadInfo info[2];
    for (int i = 0; i < 2; ++i) {
        info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, &info[i]);
    }
    for (int i = 0; i < 2; ++i) { pthread_join(threads[i], NULL); }
    return info[0].result + info[1].result;
}
```

values: global variable — shared

sum example (info struct)

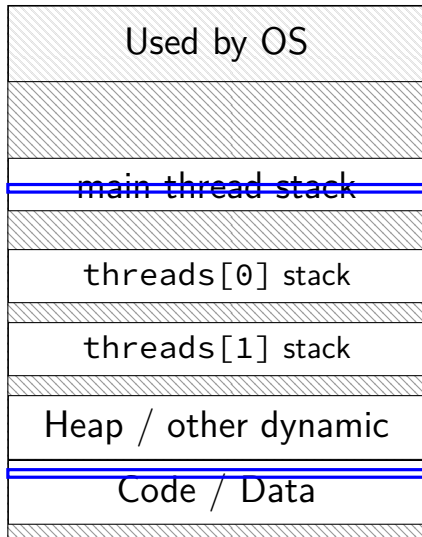
```
int values[1024];
struct ThreadInfo {
    int start, end, result;
};
void *sum_thread(void *argument) {
    struct ThreadInfo *my_info = (struct ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        sum += values[i];
    }
    my_info->result = sum;
    return NULL;
}
int sum_all() {
    pthread_t thread[2]; struct ThreadInfo info[2];
    for (int i = 0; i < 2; ++i) {
        info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&thread[i], NULL, sum_thread, &info[i]);
    }
    for (int i = 0; i < 2; ++i) { pthread_join(thread[i], NULL); }
    return info[0].result + info[1].result;
}
```

my_info: pointer to sum_all's stack;
only okay because sum_all waits!

sum example (info struct)

```
int values[1024];
struct ThreadInfo {
    int start, end, result;
};
void *sum_thread(void *argument) {
    struct ThreadInfo *my_info = (struct ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) { sum += values[i]; }
    my_info->result = sum;
    return NULL;
}
int sum_all() {
    pthread_t thread[2]; struct ThreadInfo info[2];
    for (int i = 0; i < 2; ++i) {
        info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, &info[i]);
    }
    for (int i = 0; i < 2; ++i) { pthread_join(threads[i], NULL); }
    return info[0].result + info[1].result;
}
```

thread_sum memory layout (info struct)



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

0x7F...

info array

my_info

my_info

values (global)

0x0000 0000 0040 0000

sum example (to main stack)

```
struct ThreadInfo { int *values; int start; int end; int result };
void *sum_thread(void *argument) {
    ThreadInfo *my_info = (ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        sum += my_info->values[i];
    }
    my_info->result = sum;
    return NULL;
}

int sum_all(int *values) {
    ThreadInfo info[2]; pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, (void *) &info[i]);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return info[0].result + info[1].result;
}
```

sum example (to main stack)

```
struct ThreadInfo { int *values; int start; int end; int result };
void *sum_thread(void *argument) {
    ThreadInfo *my_info = (ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        sum += my_info->values[i];
    }
    my_info->result = sum;
    return NULL;
}

int sum_all(int *values) {
    ThreadInfo info[2]; pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, (void *) &info[i]);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return info[0].result + info[1].result;
}
```

sum example (to main stack)

```
struct ThreadInfo { int *values; int start; int end; int result };
void *sum_thread(void *argument) {
    ThreadInfo *my_info = (ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        sum += my_info->values[i];
    }
    my_info->result = sum;
    return NULL;
}

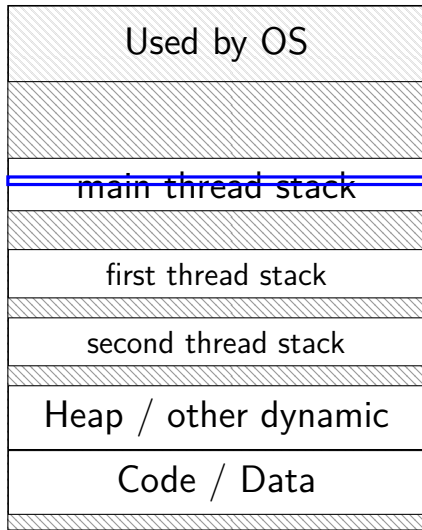
int sum_all(int *values) {
    ThreadInfo info[2]; pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, (void *) &info[i]);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return info[0].result + info[1].result;
}
```

sum example (to main stack)

```
struct ThreadInfo { int *values; int start; int end; int result };
void *sum_thread(void *argument) {
    ThreadInfo *my_info = (ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        sum += my_info->values[i];
    }
    my_info->result = sum;
    return NULL;
}

int sum_all(int *values) {
    ThreadInfo info[2]; pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, (void *) &info[i]);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return info[0].result + info[1].result;
}
```

program memory (to main stack)



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

0x7F...

info array

values (stack? heap?)

my_info

my_info

0x0000 0000 0040 0000

sum example (on heap)

```
struct ThreadInfo { pthread_t thread; int *values; int start; int end; int result;
void *sum_thread(void *argument) {
    ...
}

struct ThreadInfo *start_sum_all(int *values) {
    struct ThreadInfo *info = calloc(2, sizeof(struct ThreadInfo));
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&info[i].thread, NULL, sum_thread, (void *) &info[i]);
    }
    return info;
}

int finish_sum_all(ThreadInfo *info) {
    for (int i = 0; i < 2; ++i)
        pthread_join(info[i].thread, NULL);
    int result = info[0].result + info[1].result;
    free(info);
    return result;
}
```

sum example (on heap)

```
struct ThreadInfo { pthread_t thread; int *values; int start; int end; int result;
void *sum_thread(void *argument) {
    ...
}

struct ThreadInfo *start_sum_all(int *values) {
    struct ThreadInfo *info = calloc(2, sizeof(struct ThreadInfo));
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&info[i].thread, NULL, sum_thread, (void *) &info[i]);
    }
    return info;
}

int finish_sum_all(ThreadInfo *info) {
    for (int i = 0; i < 2; ++i)
        pthread_join(info[i].thread, NULL);
    int result = info[0].result + info[1].result;
    free(info);
    return result;
}
```

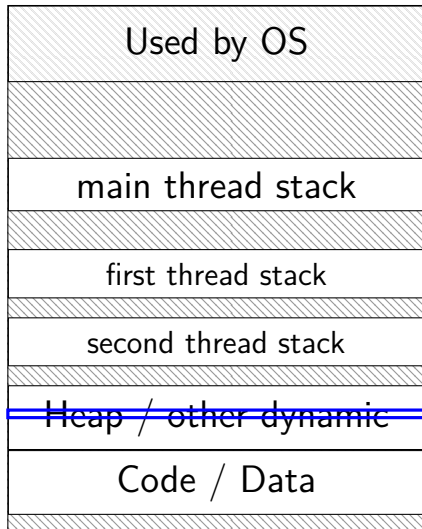
sum example (on heap)

```
struct ThreadInfo { pthread_t thread; int *values; int start; int end; int result;
void *sum_thread(void *argument) {
    ...
}

struct ThreadInfo *start_sum_all(int *values) {
    struct ThreadInfo *info = calloc(2, sizeof(struct ThreadInfo));
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&info[i].thread, NULL, sum_thread, (void *) &info[i]);
    }
    return info;
}

int finish_sum_all(ThreadInfo *info) {
    for (int i = 0; i < 2; ++i)
        pthread_join(info[i].thread, NULL);
    int result = info[0].result + info[1].result;
    free(info);
    return result;
}
```

thread_sum memory (heap version)



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

0x7F...

my_info

my_info

info array

values (stack? heap?)

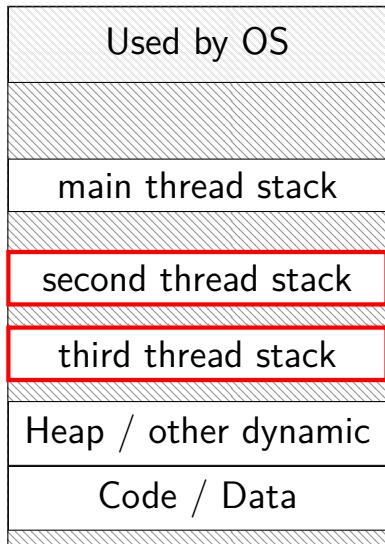
0x0000 0000 0040 0000

what's wrong with this?

```
/* omitted: headers */
void *create_string(void *ignored_argument) {
    char string[1024];
    ComputeString(string);
    return string;
}

int main() {
    pthread_t the_thread;
    pthread_create(&the_thread, NULL, create_string, NULL);
    char *string_ptr;
    pthread_join(the_thread, (void**) &string_ptr);
    printf("string_is_%s\n", string_ptr);
}
```

program memory



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

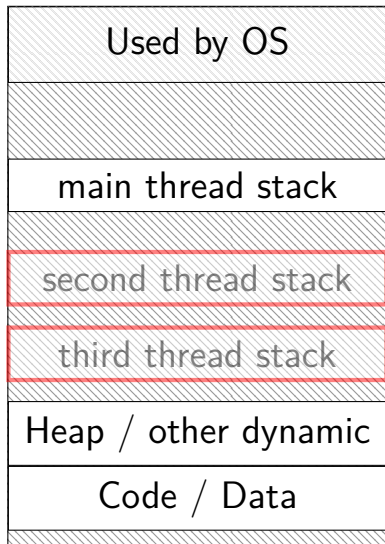
0x7F...

} dynamically allocated stacks
char string[] allocated here
string_ptr pointed to here

...stacks deallocated when
threads exit/are joined

0x0000 0000 0040 0000

program memory



0xFFFF FFFF FFFF FFFF

0xFFFF 8000 0000 0000

0x7F...

} dynamically allocated stacks
char string[] allocated here
string_ptr pointed to here

...stacks deallocated when
threads exit/are joined

0x0000 0000 0040 0000

thread joining

`pthread_join` allows collecting thread return value

if you don't join joinable thread, then **memory leak!**

thread joining

pthread_join allows collecting thread return value

if you don't join joinable thread, then **memory leak!**

avoiding memory leak?

always join...or

“detach” thread to make it not joinable

pthread_detach

```
void *show_progress(void * ...) { ... }  
void spawn_show_progress_thread() {  
    pthread_t show_progress_thread;  
    pthread_create(&show_progress_thread, NULL,  
                  show_progress, NULL);
```

/ instead of keeping pthread_t around to join thread later: */*

```
pthread_detach(show_progress_thread);
```

```
}
```

```
int main() {  
    spawn_show_progress_thread();  
    do_other_stuff();  
    ...  
}
```

detach = don't care about return value, etc.
system will deallocate when thread terminates

starting threads detached

```
void *show_progress(void * ...) { ... }  
void spawn_show_progress_thread() {  
    pthread_t show_progress_thread;  
    pthread_attr_t attrs;  
    pthread_attr_init(&attrs);  
    pthread_attr_setdetachstate(&attrs, PTHREAD_CREATE_DETACHED);  
    pthread_create(&show_progress_thread, attrs,  
                  show_progress, NULL);  
    pthread_attr_destroy(&attrs);  
}
```

setting stack sizes

```
void *show_progress(void * ...) { ... }  
void spawn_show_progress_thread() {  
    pthread_t show_progress_thread;  
    pthread_attr_t attrs;  
    pthread_attr_init(&attrs);  
    pthread_attr_setstacksize(&attrs, 32 * 1024 /* bytes */);  
    pthread_create(&show_progress_thread, attrs,  
                  show_progress, NULL);  
}
```

a threading race

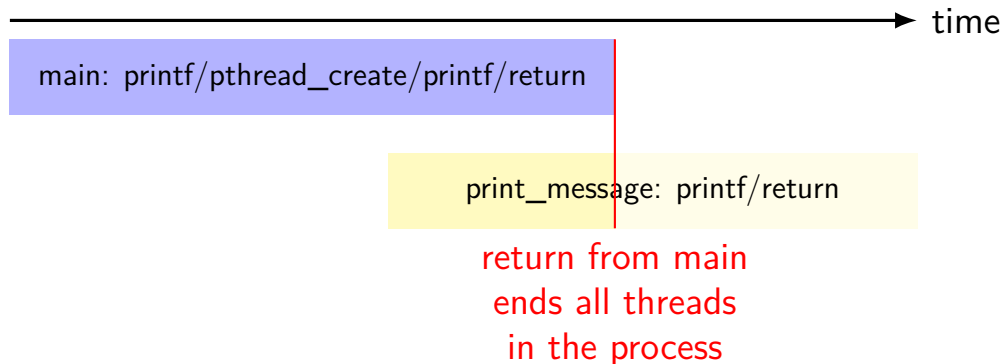
```
#include <pthread.h>
#include <stdio.h>
void *print_message(void *ignored_argument) {
    printf("In the thread\n");
    return NULL;
}
int main() {
    printf("About to start thread\n");
    pthread_t the_thread;
    /* assume does not fail */
    pthread_create(&the_thread, NULL, print_message, NULL);
    printf("Done starting thread\n");
    return 0;
}
```

My machine: outputs In the thread **about 4% of the time**.
What happened?

a race

returning from main **exits the entire process** (all its threads)
same as calling exit; not like other threads

race: main's return 0 or print_message's printf first?



the correctness problem

two threads?

introduces *non-determinism*

which one runs first?

allows for “race condition” bugs

...to be avoided with synchronization constructs

example application: ATM server

commands: withdraw, deposit

one correctness goal: don't lose money

ATM server

(pseudocode)

```
ServerLoop() {  
    while (true) {  
        ReceiveRequest(&operation, &accountNumber, &amount);  
        if (operation == DEPOSIT) {  
            Deposit(accountNumber, amount);  
        } else ...  
    }  
}  
  
Deposit(accountNumber, amount) {  
    account = GetAccount(accountNumber);  
    account->balance += amount;  
    SaveAccountUpdates(account);  
}
```

a threaded server?

```
Deposit(accountNumber, amount) {  
    account = GetAccount(accountId);  
    account->balance += amount;  
    SaveAccountUpdates(account);  
}
```

maybe GetAccount/SaveAccountUpdates can be slow?

read/write disk sometimes? contact another server sometimes?

maybe lots of requests to process?

maybe real logic has more checks than Deposit()

...

all reasons to handle multiple requests at once

→ many threads all running the server loop

multiple threads

```
main() {  
    for (int i = 0; i < NumberOfThreads; ++i) {  
        pthread_create(&server_loop_threads[i], NULL,  
                      ServerLoop, NULL);  
    }  
    ...  
}  
  
ServerLoop() {  
    while (true) {  
        ReceiveRequest(&operation, &accountNumber, &amount);  
        if (operation == DEPOSIT) {  
            Deposit(accountNumber, amount);  
        } else ...  
    }  
}
```

the lost write

account->balance += amount; (in two threads, same account)

Thread A

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

context switch

```
mov %rax, account->balance
```

the lost write

account->balance += amount; (in two threads, same account)

Thread A

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

lost write to balance

Thread B

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

“winner” of the race

the lost write

account->balance += amount; (in two threads, same account)

Thread A

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

lost write to balance

lost track of thread A's money

Thread B

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

“winner” of the race

thinking about race conditions (1)

what are the possible values of x ? (initially $x = y = 0$)

Thread A	Thread B
$x \leftarrow 1$	$y \leftarrow 2$

thinking about race conditions (2)

possible values of x ? (initially $x = y = 0$)

Thread A	Thread B
-----------------	-----------------

$x \leftarrow y + 1$	$y \leftarrow 2$
	$y \leftarrow y \times 2$

thinking about race conditions (2)

possible values of x ? (initially $x = y = 0$)

Thread A	Thread B
$x \leftarrow y + 1$	$y \leftarrow 2$
	$y \leftarrow y \times 2$

thinking about race conditions (3)

what are the possible values of x ?

(initially $x = y = 0$)

Thread A	Thread B
$x \leftarrow 1$	$x \leftarrow 2$

thinking about race conditions (2)

possible values of x ? (initially $x = y = 0$)

Thread A	Thread B
$x \leftarrow y + 1$	$y \leftarrow 2$
	$y \leftarrow y \times 2$

atomic operation

atomic operation = operation that runs to completion or not at all

we will use these to let threads work together

most machines: loading/storing (aligned) words is atomic

so can't get 3 from $x \leftarrow 1$ and $x \leftarrow 2$ running in parallel

aligned $\approx$ address of word is multiple of word size (typically done by compilers)

but some instructions are not atomic; examples:

x86: integer add constant to memory location

many CPUs: loading/storing values that cross cache blocks

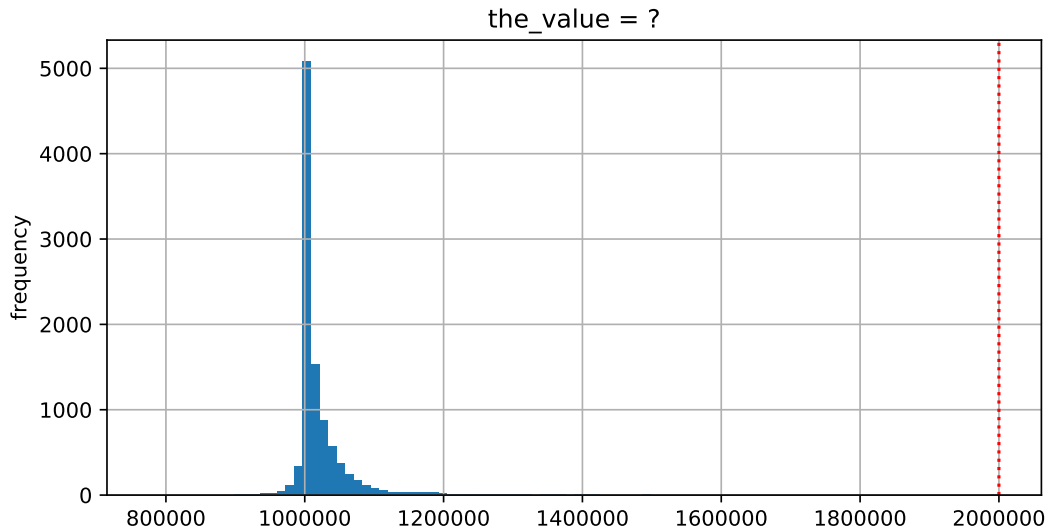
e.g. if cache blocks 0x40 bytes, load/store 4 byte from addr. 0x3E is not atomic

lost adds (program)

```
.global update_loop
update_loop:
    addl $1, the_value // the_value (global variable) += 1
    dec %rdi           // argument 1 -= 1
    jg update_loop     // if argument 1 >= 0 repeat
    ret
```

```
int the_value;
extern void *update_loop(void *);
int main(void) {
    the_value = 0;
    pthread_t A, B;
    pthread_create(&A, NULL, update_loop, (void*) 1000000);
    pthread_create(&B, NULL, update_loop, (void*) 1000000);
    pthread_join(A, NULL); pthread_join(B, NULL);
    // expected result: 1000000 + 1000000 = 2000000
    printf("the_value = %d\n", the_value);
}
```

lost adds (results)



but how?

probably not possible on single core

exceptions can't occur in the middle of add instruction

...but 'add to memory' implemented with multiple steps

still needs to load, add, store internally

can be interleaved with what other cores do

but how?

probably not possible on single core

exceptions can't occur in the middle of add instruction

...but 'add to memory' implemented with multiple steps

still needs to load, add, store internally

can be interleaved with what other cores do

(and actually it's more complicated than that — we'll talk later)

so, what is actually atomic

for now we'll assume: load/stores of 'words'
(64-bit machine = 64-bits words)

in general: processor designer will tell you

their job to design caches, etc. to work as documented

compilers move loads/stores (1)

```
void WaitForReady() {  
    do {} while (!ready);  
}
```

```
WaitForOther:  
    movl ready, %eax    // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2              // while (eax == 0) repeat  
    ...
```

compilers move loads/stores (1)

```
void WaitForReady() {  
    do {} while (!ready);  
}
```

```
WaitForOther:  
    movl ready, %eax    // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2              // while (eax == 0) repeat  
    ...
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
    // compiler optimization: don't set is_waiting to 1,  
    // (why? it will be set to 0 anyway)  
    movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
    // compiler optimization: don't set is_waiting to 1,  
    // (why? it will be set to 0 anyway)  
    movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

compilers move loads/stores (2)

```
void WaitForOther() {  
    is_waiting = 1;  
    do {} while (!other_ready);  
    is_waiting = 0;  
}
```

WaitForOther:

```
// compiler optimization: don't set is_waiting to 1,  
// (why? it will be set to 0 anyway)  
movl other_ready, %eax // eax <- other_ready  
.L2:  
    testl %eax, %eax  
    je .L2 // while (eax == 0) repeat  
    ...  
    movl $0, is_waiting // is_waiting <- 0
```

fixing compiler reordering?

isn't there a way to tell compiler not to do these optimizations?

yes, but that is **still not enough!**

processors sometimes do this kind of reordering too (between cores)

pthread and reordering

many pthreads functions **prevent reordering**

everything before function call actually happens before

includes **preventing some optimizations**

e.g. keeping global variable in register for too long

pthread_create, pthread_join, other tools we'll talk about ...

basically: if pthreads is waiting for/starting something, no weird ordering

implementation part 1: prevent compiler reordering

implementation part 2: use special instructions

example: x86 mfence instruction

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

critical section: code that exactly one thread can execute at a time

result of mutual exclusion

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time

like updating shared balance

critical section: code that exactly one thread can execute at a time

result of mutual exclusion

lock: object only one thread can hold at a time

interface for creating critical sections

lock analogy

agreement: only change account balances while wearing this hat

normally hat kept on table

put on hat when editing balance

hopefully, only one person (= thread) can wear hat a time

need to wait for them to remove hat to put it on

lock analogy

agreement: only change account balances while wearing this hat

normally hat kept on table

put on hat when editing balance

hopefully, only one person (= thread) can wear hat a time

need to wait for them to remove hat to put it on

“lock (or acquire) the lock” = get and put on hat

“unlock (or release) the lock” = put hat back on table

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — wait until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — **wait** until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

waiting for lock?

when waiting — ideally:

not using processor (at least if waiting a while)

OS can context switch to other programs

pthread mutex

```
#include <pthread.h>
```

```
pthread_mutex_t account_lock;  
pthread_mutex_init(&account_lock, NULL);  
    // or: pthread_mutex_t account_lock =  
    //      PTHREAD_MUTEX_INITIALIZER;  
...  
pthread_mutex_lock(&account_lock);  
balance += ...;  
pthread_mutex_unlock(&account_lock);
```

exercise

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

exercise (alternate 1)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

exercise (alternate 2)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init_one", two = "init_two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one_in_ThreadB"; // (B1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two_in_ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
}
```

possible values of one/two after A+B run?

POSIX mutex restrictions

pthread_mutex rule: unlock from same thread you lock in

does this actually matter?

depends on how pthread_mutex is implemented

preview: general sync

lots of coordinating threads beyond locks

will talk about two general tools later:k

- monitors/condition variables

- semaphores [if time]

big added feature: wait for arbitrary thing to happen

also some less general tools: barriers

a bad idea

one **bad** idea to wait for an event:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; bool ready = false;
void WaitForReady() {
    pthread_mutex_lock(&lock);
    do {
        pthread_mutex_unlock(&lock);
        /* only time MarkReady() can run */
        pthread_mutex_lock(&lock);
    } while (!ready);
    pthread_mutex_unlock(&lock);
}
void MarkReady() {
    pthread_mutex_lock(&lock);
    ready = true;
    pthread_mutex_unlock(&lock);
}
```

wastes processor time; MarkReady can stall waiting for unlock window

beyond locks

in practice: want more than locks for synchronization

for waiting for arbitrary events (without CPU-hogging-loop):

- monitors

- semaphores

for common synchronization patterns:

- barriers

- reader-writer locks

higher-level interface:

- transactions

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU

one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU
one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers API

`barrier.Initialize(NumberOfThreads)`

`barrier.Wait()` — return after all threads have waited

idea: multiple threads perform computations in parallel

threads wait for **all other threads** to call `Wait()`

barrier: waiting for finish

```
barrier.Initialize(2);
```

Thread 0

```
partial_mins[0] =  
    /* min of first  
       50M elems */;
```

```
barrier.Wait();
```

```
total_min = min(  
    partial_mins[0],  
    partial_mins[1]  
);
```

Thread 1

```
partial_mins[1] =  
    /* min of last  
       50M elems */  
barrier.Wait();
```

barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

barriers: reuse

Thread 0

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

Thread 1

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

pthread barriers

```
pthread_barrier_t barrier;  
pthread_barrier_init(  
    &barrier,  
    NULL /* attributes */,  
    numberOfThreads  
);  
...  
...  
pthread_barrier_wait(&barrier);
```

exercise

```
pthread_barrier_t barrier; int x = 0, y = 0;
void thread_one() {
    y = 10;
    pthread_barrier_wait(&barrier);
    y = x + y;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    printf("%d_ %d\n", x, y);
}
void thread_two() {
    x = 20;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    x = x + y;
    pthread_barrier_wait(&barrier);
}
```

output? (if both run at once, barrier set for 2 threads)

life homework (pseudocode)

```
for (int time = 0; time < MAX_ITERATIONS; ++time) {  
    for (int y = 0; y < size; ++y) {  
        for (int x = 0; x < size; ++x) {  
            to_grid(x, y) = computeValue(from_grid, x, y);  
        }  
    }  
    swap(from_grid, to_grid);  
}
```

life homework

compute grid of values for time t from grid for time $t - 1$

compute new value at i, j based on surrounding values

parallel version: produce parts of grid in different threads

use barriers to finish time t before going to time $t + 1$

backup slides